1ª Avaliação de JCRPGPD – Programação Paralela e Distribuída

Tema: Análise Paralela de Transações Financeiras utilizando `ExecutorService` e Afinidade de Processador

1. Objetivo

Desenvolver, em Java, uma aplicação para análise de grandes volumes de transações financeiras utilizando técnicas de programação paralela baseada em **paralelismo de dados**.

A aplicação deverá processar um conjunto de transações financeiras, comparando uma implementação sequencial com implementações paralelas utilizando `ExecutorService`, com e sem afinidade de processador.

O trabalho deverá ser realizado em grupos de **até 4 alunos**.

---

2. Contextualização

Uma empresa possui um grande conjunto de transações financeiras realizadas por seus clientes.

Cada transação poderá possuir informações como:

```
id
cliente
data
valor
categoria
cidade
estado
tipo
...
```

Exemplo:

```
ID       Cliente    Valor      Categoria
10001    C001       350,50     Alimentação
10002    C025       1200,00    Eletrônicos
10003    C018       85,90      Transporte
10004    C001       2500,00    Eletrônicos
...
```

Considere um conjunto contendo:

- 1 milhão;

- 10 milhões;
- 50 milhões de transações.
- 100 milhões de transações.
- 500 milhões de transações

Os dados devem ser gerados de **forma sintética** e contendo **diferentes distribuições estatísticas** (no mínimo quatro distribuições).

O sistema deverá realizar diferentes análises sobre esse conjunto.

Como as transações podem ser analisadas independentemente em diversas operações, o problema apresenta características adequadas ao **paralelismo de dados**.

---

3. Problema Computacional

O programa deverá processar um grande conjunto de transações e realizar diferentes análises.

Entre as análises possíveis estão:

Análise 1 — Soma dos valores

Calcular o valor total das transações.

Análise 2 — Média

Calcular o valor médio das transações:

Análise 3 — Detecção de transações suspeitas

Identificar transações que atendam a determinados critérios de transações suspeitas definidas pelos desenvolvedores.

Por exemplo:

```
valor > R$ 10.000
```

ou:

```
valor > R$ 5.000
e
categoria = "Eletrônicos"
```

Análise 4 — Estatísticas por categoria

Calcular:

```
Alimentação       → quantidade / valor total
Transporte        → quantidade / valor total
Eletrônicos       → quantidade / valor total
Saúde             → quantidade / valor total
Educação          → quantidade / valor total
```

Análise 5 — Outras análises estatísticas definidas pelos desenvolvedores

---

4. Paralelização dos Dados

O conjunto de transações deverá ser dividido em partes.

Por exemplo, considerando:

```
10.000.000 transações
```

e:

```
4 threads
```

O programa poderá dividir os dados em:

```
Thread 1 → transações 0 até 2.499.999
Thread 2 → transações 2.500.000 até 4.999.999
Thread 3 → transações 5.000.000 até 7.499.999
Thread 4 → transações 7.500.000 até 9.999.999
```

Cada thread executará o mesmo algoritmo sobre uma parte diferente dos dados.

Isso caracteriza **paralelismo de dados**.

---

5. ExecutorService

A implementação paralela deverá obrigatoriamente utilizar:

```
ExecutorService
```

Por exemplo:

```
ExecutorService executor =
        Executors.newFixedThreadPool(numeroThreads);
```

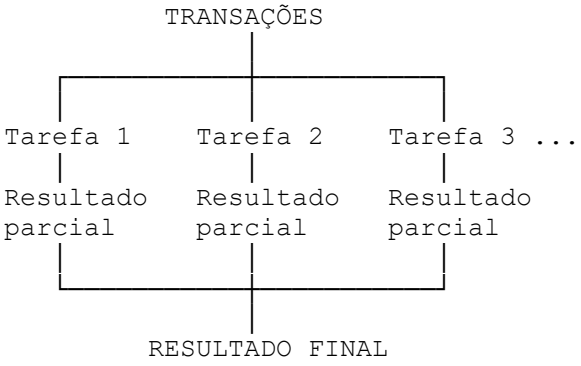
As partes do conjunto de dados deverão ser transformadas em tarefas.

Cada tarefa poderá ser implementada utilizando:

```
Callable<T>
```

Permitindo retornar os resultados parciais.

Por exemplo:



---

6. Combinação dos Resultados

Cada thread deverá produzir um resultado parcial.

Por exemplo:

```
Thread 1 → R$ 2.500.000
Thread 2 → R$ 2.300.000
Thread 3 → R$ 2.700.000
Thread 4 → R$ 2.500.000
```

O programa deverá combinar os resultados:

```
Total = R$ 10.000.000
```

Essa etapa permitirá trabalhar com um conceito importante:

Redução de resultados paralelos.

---

7. Afinidade de Processador

O projeto deverá utilizar uma biblioteca de afinidade de processador, como a **OpenHFT Affinity**.

O grupo deverá realizar experimentos nos quais as threads sejam associadas a processadores específicos.

Exemplo:

```
Thread 1 → CPU 0
Thread 2 → CPU 1
Thread 3 → CPU 2
Thread 4 → CPU 3
```

Deverá ser realizada uma comparação entre:

```
ExecutorService
        ↓
sem afinidade
```

e:

```
ExecutorService
        ↓
com afinidade
```

O objetivo é verificar experimentalmente o impacto dessa configuração no processamento.

---

8. Divisão das Atividades entre os Alunos

O projeto deverá possuir atividades individuais claramente identificáveis.

Aluno 1 — Agregação financeira

Será responsável por implementar:

- soma dos valores;
- média;
- maior transação;
- menor transação.
- ...

Deverá produzir:

```
versão sequencial
versão ExecutorService
versão ExecutorService + Affinity
```

Aluno 2 — Detecção de transações suspeitas

Será responsável pela identificação de transações que atendam a determinados critérios.

Por exemplo:

```
valor > R$ 10.000
```

e:

```
valor > R$ 5.000
categoria = Eletrônicos
```

As categorias de transações suspeitas deverão ser criadas arbitrariamente.

O aluno deverá comparar o processamento sequencial e paralelo.

Aluno 3 — Estatísticas por categoria

Será responsável por calcular as diversas estatísticas:

```
Categoria
Quantidade
Valor total
Valor médio
Maior transação
...
```

Por exemplo:

```
Categoria       Quantidade       Total
-----------------------------------------------
Alimentação     2.500.000        R$ ...
Transporte      1.800.000        R$ ...
Saúde           1.200.000        R$ ...
Eletrônicos     1.500.000        R$ ...
Educação          900.000        R$ ...
```

Aluno 4 — Benchmark e análise de desempenho

Será responsável por desenvolver o módulo de experimentação.

Deverá medir:

- tempo sequencial;
- tempo paralelo;
- tempo com afinidade;
- speedup;
- eficiência paralela.

Também deverá produzir **gráficos** e **tabelas** dos resultados.

---

9. Experimentos Obrigatórios

Os testes deverão ser realizados com diferentes quantidades de threads.

Por exemplo:

```
1 thread
2 threads
4 threads
8 threads
```

Para cada configuração deverão ser comparados:

```
Sequencial
ExecutorService
ExecutorService + Affinity
```

---

10. Diferentes Tamanhos de Dados

O grupo deverá utilizar diferentes volumes de transações.

Sugestão:

```
100.000 transações
1.000.000 transações
10.000.000 transações
50.000.000 transações
250.000.000 transações
500.000.000 transações
...
```

Caso o computador utilizado não suporte o maior volume, o grupo deverá utilizar o maior conjunto que possa ser processado adequadamente.

---

11. Métricas

Para cada experimento deverão ser registrados:

Configuração	Threads	Dados	Tempo
Sequencial	1	1 milhão	...
Executor	2	1 milhão	...
Executor	4	1 milhão	...
Executor + Affinity	2	1 milhão	...
Executor + Affinity	4	1 milhão	...

Também deverão ser calculados:

Speedup

Eficiência Paralela

---

12. Questão Central do Projeto

O grupo deverá investigar experimentalmente a seguinte questão:

Qual é o impacto do uso de `ExecutorService` e da afinidade de processador no desempenho da análise de grandes volumes de transações financeiras?

A resposta deverá ser baseada nos resultados obtidos pelos próprios alunos.

O grupo deverá discutir:

- quantidade de threads;
- quantidade de processadores;
- tamanho do conjunto de dados;
- overhead;
- balanceamento de carga;
- speedup;
- eficiência;
- afinidade de processador;
- escalabilidade.

---

13. Requisito Importante

O programa **não poderá simplesmente criar uma thread para cada transação**.

Por exemplo, para:

```
10.000.000 transações
```

não deverá ser criado:

```
10.000.000 threads
```

O conjunto deverá ser dividido em um número adequado de tarefas.

Os alunos deverão justificar a quantidade de tarefas utilizadas.

---

14. Relatório

O relatório deverá conter:

1. Introdução;
2. Descrição do problema;
3. Caracterização do paralelismo de dados;
4. Estrutura do conjunto de dados;
5. Algoritmo sequencial;
6. Estratégia de divisão dos dados;
7. Implementação com `ExecutorService`;
8. Implementação da afinidade de processador;
9. Metodologia dos experimentos;
10. Resultados;
11. Speedup;
12. Eficiência Paralela;
13. Análise dos resultados;
14. Conclusão;
15. Referências.

---

15. Avaliação individual

Apesar de o projeto ser desenvolvido em grupo, cada aluno deverá possuir uma contribuição individual claramente identificável.

Cada integrante deverá:

- implementar sua funcionalidade;
- apresentar o código desenvolvido;
- explicar sua estratégia de paralelização;
- realizar testes;
- participar da integração;
- explicar os resultados;
- responder individualmente às perguntas sobre sua implementação.

Durante a apresentação, o professor poderá solicitar pequenas modificações no código para verificar o domínio individual do aluno.

---

16. Entregas

O grupo deverá entregar:

1. Código-fonte;
2. Arquivo de configuração do projeto;
3. Conjunto de dados ou mecanismo para sua geração;
4. Relatório técnico;
5. Resultados dos experimentos;
6. Gráficos;
7. Identificação da contribuição individual de cada integrante.

O relatório deverá apresentar uma seção denominada **"Contribuição Individual"**, especificando exatamente o que foi desenvolvido por cada integrante.

---

17. Critérios de Avaliação

Critério	Peso
Implementação sequencial	10%
Paralelização com ExecutorService	20%
Afinidade de processador	15%
Divisão dos dados	10%
Benchmark	15%
Speedup e eficiência	10%
Relatório	10%
Defesa individual	10%
Total	**100%**

A avaliação individual deverá considerar tanto a contribuição efetivamente desenvolvida pelo aluno quanto sua capacidade de explicar e modificar o próprio código.

---

18. Resultado Esperado

Ao final, o sistema deverá produzir resultados semelhantes a:

```
==============================================
 ANÁLISE PARALELA DE TRANSAÇÕES FINANCEIRAS
==============================================

Quantidade de transações: 10.000.000

---------------- SEQUENCIAL -----------------

Tempo: 8.521 ms

---------------- EXECUTOR -------------------

Threads: 2
```

```
Tempo: 4.632 ms
Speedup: 1,84
Eficiência: 92,0%

Threads: 4
Tempo: 2.381 ms
Speedup: 3,58
Eficiência: 89,5%

Threads: 8
Tempo: 1.954 ms
Speedup: 4,36
Eficiência: 54,5%

---------- EXECUTOR + AFFINITY -------------

Threads: 2
CPUs: 0,1
Tempo: ...

Threads: 4
CPUs: 0,1,2,3
Tempo: ...

Threads: 8
CPUs: 0,1,2,3,4,5,6,7
Tempo: ...
```

Os valores acima são apenas exemplos. Os alunos deverão apresentar os resultados obtidos em seus próprios computadores e em formatos de **tabelas** e **gráficos**.